

python概述

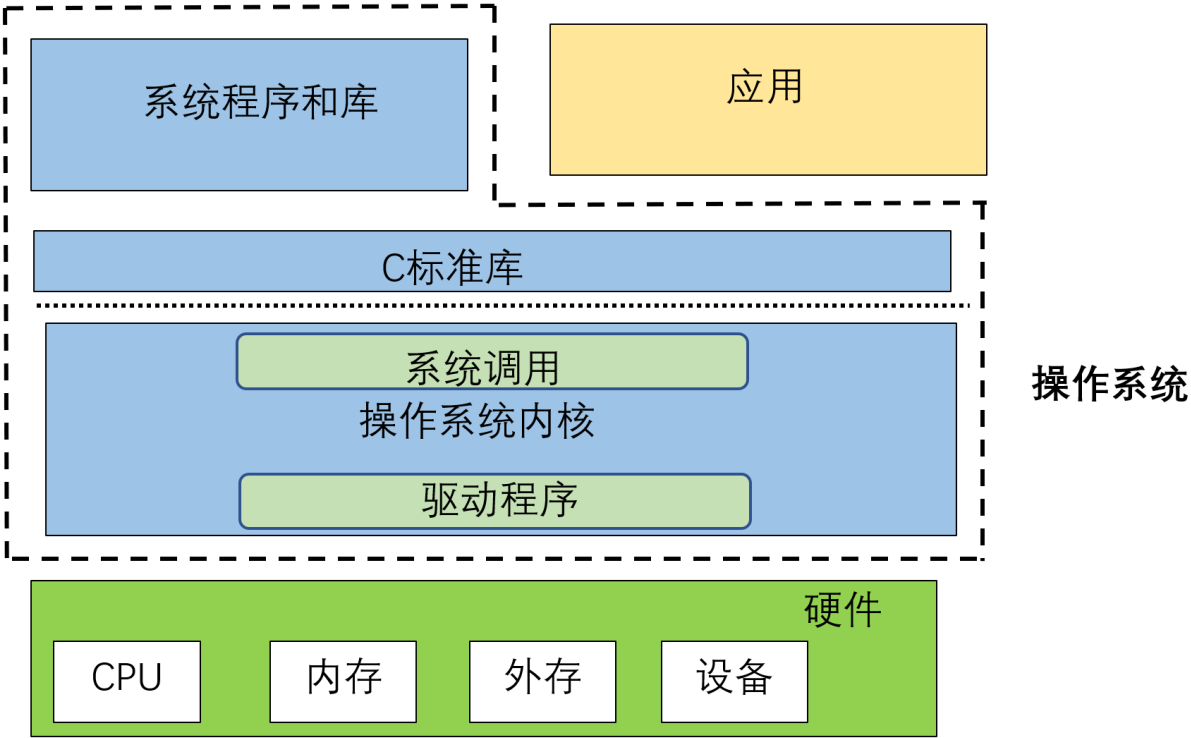
主要内容

- 程序设计语言
- Python程序的运行
- 对象基础
- 模块

计算机概述：硬件和软件

- 硬件(hardware): 组成计算机的各个物理部件
- **软件(software)**: 要执行的（二进制）**程序**、相应的**数据**以及**文档**组合在一起
 - 程序(program): 一系列指令，计算机根据指令进行运算完成相应的任务
- 操作系统(Operating System):
 - 管理和控制计算机系统中的硬件和软件资源，提供一个友好的用户界面以方便使用者操作计算机
 - **内核 (kernel)** 是操作系统的核心，开机时启动并一直运行
 - 操作系统=内核+**系统程序和库**

开发商	台式机	笔记本电脑	平板电脑	智能手机
微软公司	视窗(Windows)			Windows Phone
苹果公司	Mac OS		iOS	
开源软件	Linux			
谷歌公司	/	/	安卓(Android)	
华为公司	鸿蒙(HarmonyOS)			

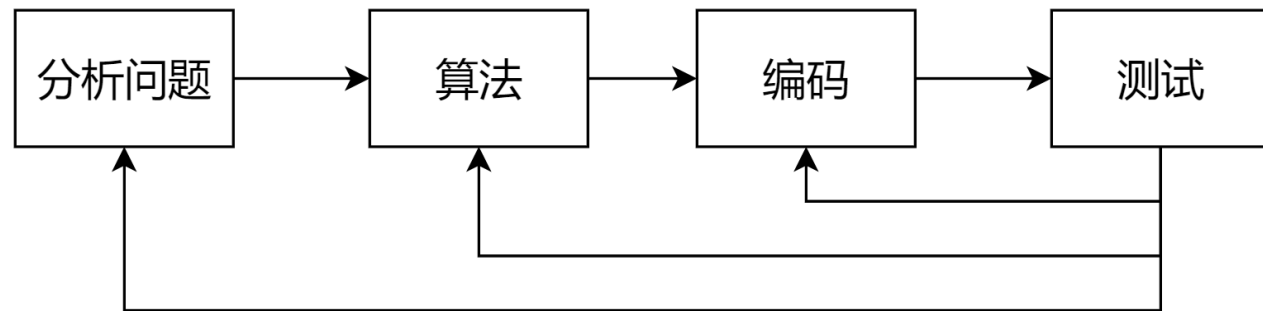


编程(Programming)

- 程序(program): 计算机可以执行的一系列指令/语句(instruction/statement)或代码(code)
 - 程序执行: 执行程序中包含的指令
- 程序语言: 描述这些指令或语句的语言, 是“程序员”与“机器”对话的语言
 - 语法 (syntax): 哪些符号或文字的组合方式是正确的
 - 语义 (semantics): 描述程序的意义, 当代码运行时计算机干什么
- 算法(algorithm)
 - 描述了为了解决某个问题需要采取的动作以及这些动作的顺序
 - 自然语言、伪代码 (pseudo-code, 自然语言混杂着一些程序代码)、程序流程图的形式
- 编程(programming): 为了解决某个问题, 设计算法, 然后将其转换为采用某种程序语言描述的程序



程序语言



低级程序语言

- 机器语言 (Machine code) :

- 计算机的运算采用二进制:
 - 所有程序和数据以二进制方式存储
 - 位/比特(bit): 单个二进制数字, 可取值1或者0
 - 字节(byte): 8个连续的比特
- 以数字的形式 (十六进制) 描述了CPU所执行的指令, 比如 05 01 23
- 编写的代码称为**目标代码** (机器代码)

- 汇编语言 (Assembly Language) : 以人可读的方式描述机器代码

```
add EAX, 1  
mov [ESP+4], EAX
```

- 不同架构的CPU有不同的机器代码: Intel (32和64比特)、ARM、PowerPC等
- 经常需要大量的指令才能完成某个功能
- 难学、容易出错

汇编程序

```
add EAX, 1  
mov [ESP+4], EAX  
...
```

汇编器
Assembler



机器代码

```
10100001  
10110010  
...
```

高级程序语言

- 类自然语言，不再依赖某种特定的机器或环境。在不同的平台上会被转换成不同的机器语言
- **语句(statement)**：相当于指令，由关键字、运算符、标点符号以及其他编程元素（字面量等）按照相应的顺序组成
- 按照高级语言编写的程序称为源程序或源代码（source code）
- 更加容易学习和使用，一个语句经常对应着许多条机器代码
- Fortran, C, C++, C#, Java, Javascript, Python, Visual Basic, Ruby, Go, Rust等

```
total = price * (tax + 100) / 100
```



```
mov EAX, EBP[-2]  
mov EBX, EBP[-4]  
add EBX, 100  
mul EAX, EBX  
div EAX, 100  
mov EBP[2], EAX
```

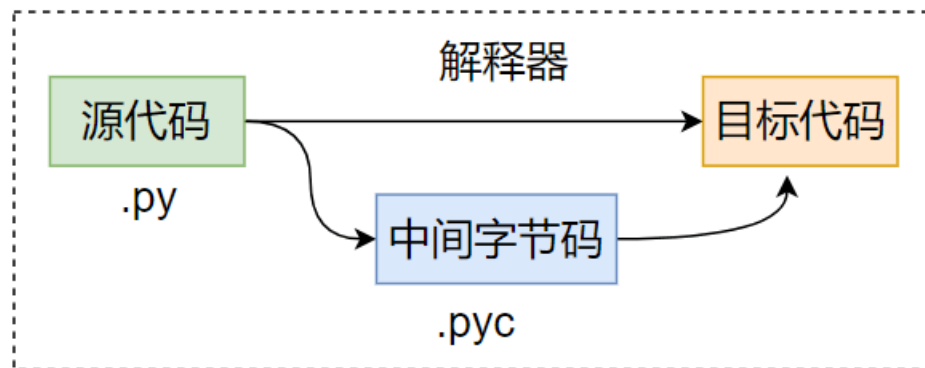
解释和编译

如何将使用**高级语言编写的程序（源代码/源程序source code）**转换为**目标代码（机器码）**？

- **解释器（Interpreter）**：将语句翻译成机器码，**并且执行**，Python、bash、Javascript、Perl、PhP等
 - 修改程序方便，修改代码重新运行就可以了
 - 必须有解释器才可以运行，跨平台（只要有解释器，就可以执行）
 - 每次运行，都要进行翻译，运行速度会有影响
- **编译器（Compiler）**：将语句翻译成机器码，**形成目标代码文件**(机器语言)，C、C++、VB等
 - 修改程序后需要进行编译
 - 编译一次，编译后的目标代码可以直接在相应的操作系统中运行，不再需要编译器
 - 编译时相比解释可以作更多的优化
- 有些语言将解释和编译结合在一起
 - Python语言：解释器(自动) 首先将.py程序转换（编译）为.pyc字节码，然后由解释器解释执行
 - **被import的模块源程序**，还会保存之前转换的.pyc字节码，源程序没有修改时可省略转换过程，优化程序和提高执行速度
 - Python社区正在优化运行速度

解释和编译

```
$ cat hello.py
print('hello world!' )
$ python hello.py
hello world!
```



开发环境和运行环境

.py

.pyc

Python解释器平台

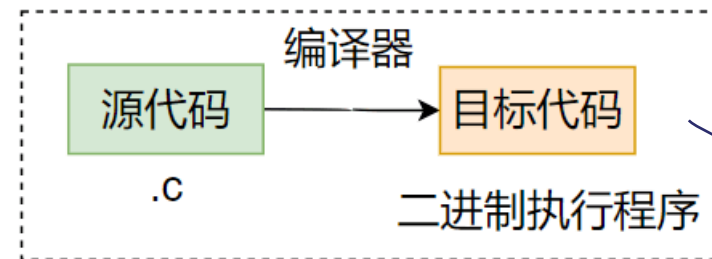
Windows

Linux

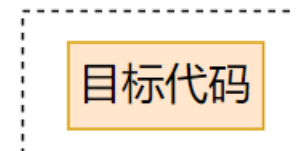
MacOS

Android

IOS



开发环境



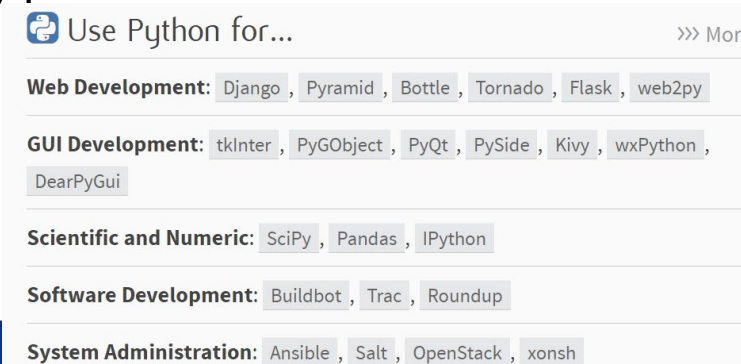
运行环境

```
$ cat hello.c
#include <stdio.h>

int main(char *argv[])
{
    printf("Hello world!\n");
}
$ gcc hello.c
$ ./a.out
hello world!
```



- 1989年吉多·范罗苏姆 [Guido van Rossum](#) 发明Python [ˈpaɪθən]语言
- **简单、易学**：Python是一种代表简单主义思想的语言
- **开源、免费**：Python是FLOSS（Free/Libre and Open Source Software，自由/开放源码软件）之一。使用者可以自由地发布这个软件的拷贝、阅读它的源代码、对它做改动、把它的一部分用于新的自由软件中
- **跨平台**：所编写程序在解释器支持下可无需修改在Windows、Linux、Mac等操作系统上使用
- **灵活性**：Python支持多种编程范式，包括过程式编程、面向对象编程、函数式编程
- **可扩展和可嵌入性**：胶水语言，支持采用C、C++等语言编写扩充模块，也可为C、C++程序提供脚本功能
- **丰富的扩展库支持**：拥有大量的几乎支持所有领域应用开发的成熟扩展库
- 可广泛应用在Web应用、图形用户界面开发、系统管理、科学计算、人工智能、大数据、游戏等

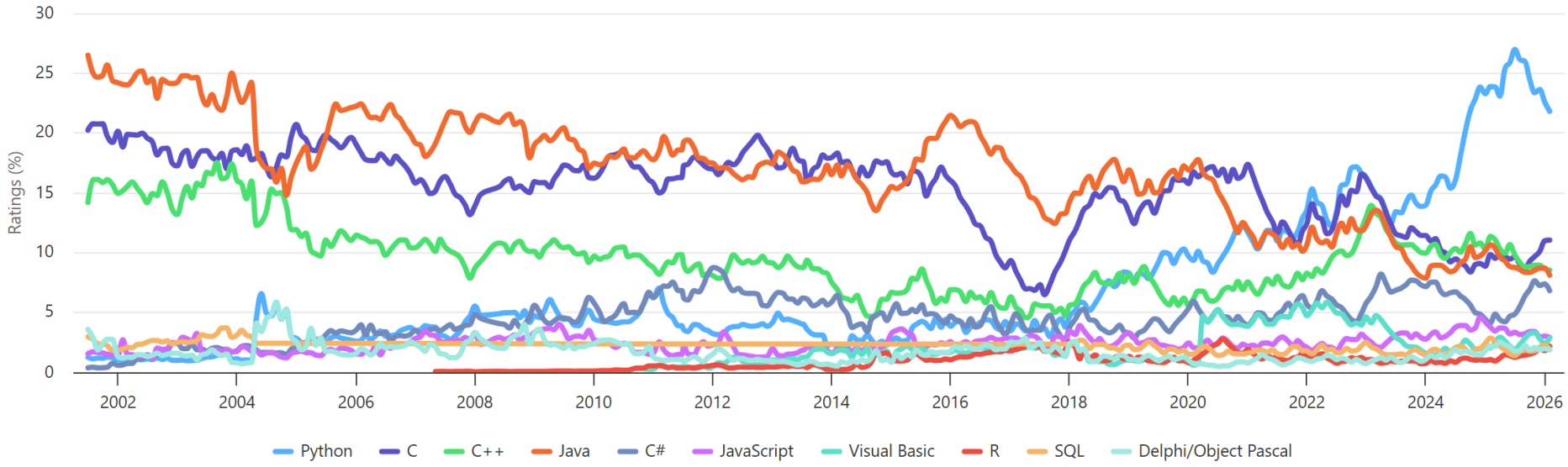


Python语言

- TIOBE编程语言排行榜 (<https://www.tiobe.com/tiobe-index>)：每月更新一次，反映了编程语言的欢迎程度

Feb 2026	Feb 2025	Change	Programming Language		Ratings	Change
1	1			Python	21.81%	-2.08%
2	4	▲		C	11.05%	+1.22%
3	2	▼		C++	8.55%	-2.82%
4	3	▼		Java	8.12%	-2.54%
5	5			C#	6.83%	+2.71%

6	6
7	10
8	15
9	7
10	9



主要内容

- 程序设计语言
- **Python程序的运行**
- 对象基础
- 模块

Python版本

我们的教学采用Python3：最初于2008年12月发布。最新版本为2026年2月3日发布的3.14.3

- 可同时安装多个版本，通过菜单选择不同的Python版本。 <http://www.python.org/downloads/>
- standalone installer for Python3.14.3：安装python 3.14.3
- Python install manager：安装python3.14.3 + python install manager(python版本维护安装工具)
 - py help 查看相应帮助



Python版本

- 安装的python平台包括了：
 - (交互式的) python解释器(interpreter)
 - (图形化的) IDLE：集成开发环境IDE (Integrated Development Environment)
 - 将编辑器、调试器、解释器环境综合在一起
 - 其他常用的IDE还包括PyCharm、**Visual Studio Code(建议)**、Spyder、Jupyter等

- **Online Python - IDE, Editor, Compiler, Interpreter:**
<https://www.online-python.com>
- python代码可视化运行: <https://pythontutor.com>

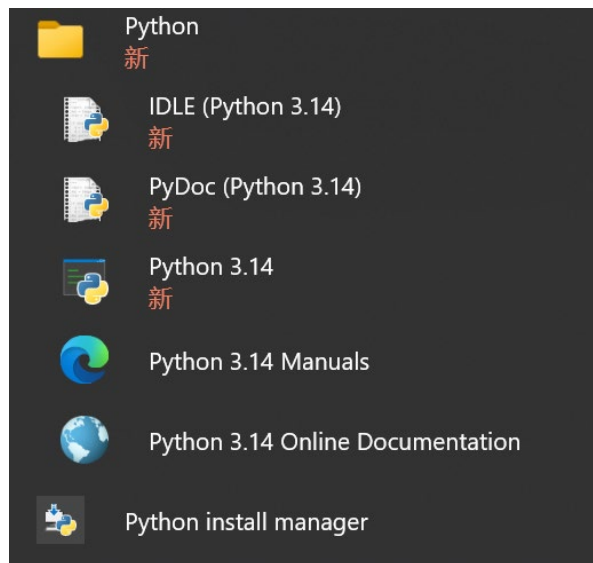


集成开发环境

解释器

Python手册

模块手册



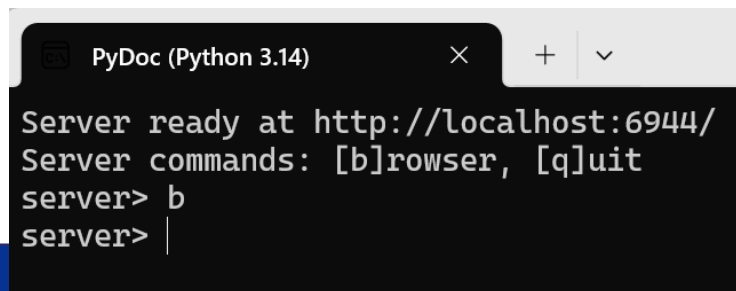
* 集成开发环境

* 启动HTTP服务器, 提供各个模块文档

* 解释器

* Python手册

* <https://docs.python.org/3.14/>

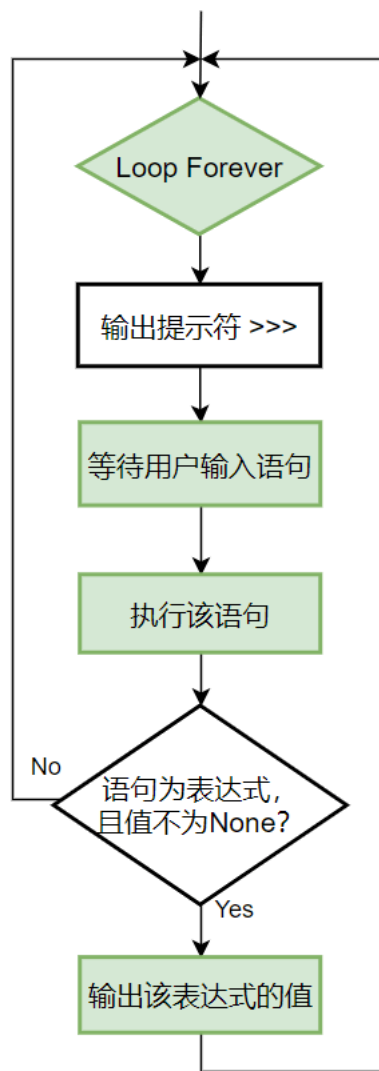


执行python代码的两种模式：交互式和脚本模式

- 交互式：Read-Evaluate-(if expression and not None) Print Loop (REPL)

- Python解释器的提示符为 >>>
- 可以当成计算器来使用

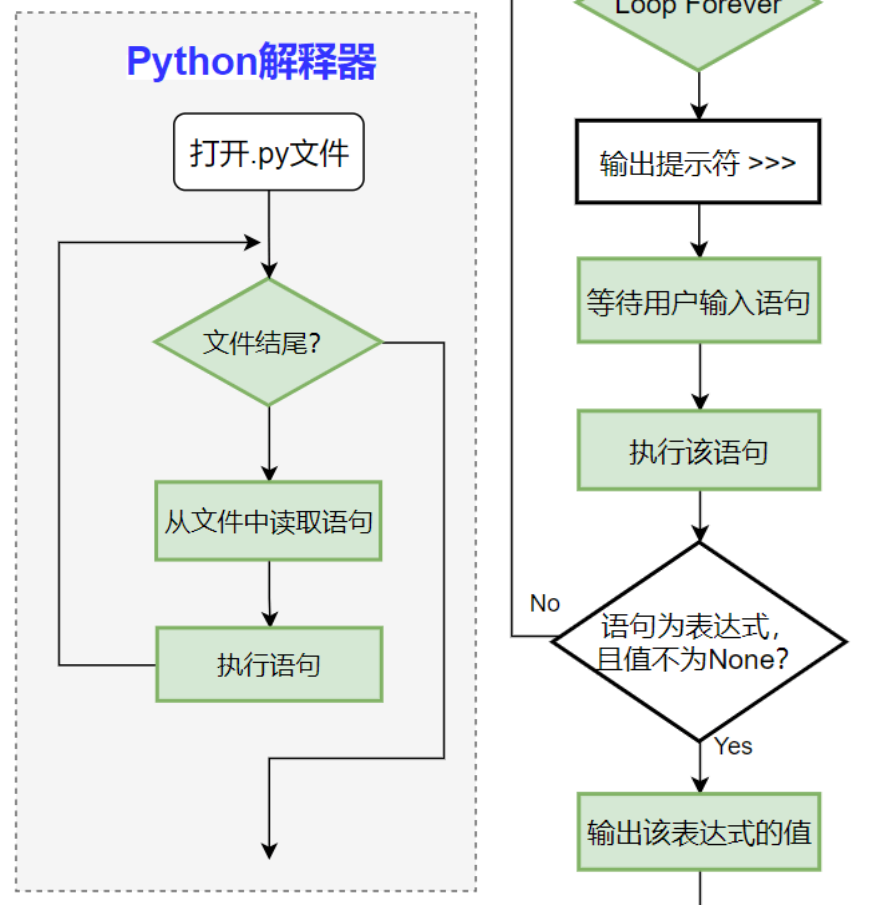
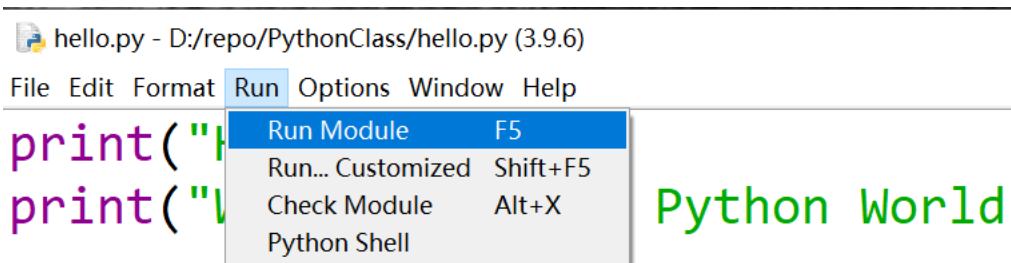
```
IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59,
AMD64) on win32
Type "help", "copyright", "credits" o
>>> 4 + 5
9
>>> print('hello world!')
hello world!
>>> |
```



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28
Type "help", "copyright", "credits" or "li
>>> 3
3
>>> 4 + 5
9
>>> 1 - 7
-6
>>> "word"
'word'
>>> word
Traceback (most recent call last):
  File "<pysHELL#4>", line 1, in <module>
    word
NameError: name 'word' is not defined
>>> print("hello world!")
hello world!
>>> |
```

执行python代码的两种模式：交互式和脚本模式

- 脚本(script)方式：新建文件 或者 打开文件 + 运行
 - 首先通过编辑器或集成开发环境编写python源程序(也称为python脚本)，该源程序包括多条python语句
 - Run子菜单→Run Module(F5)运行**，可以在Python Shell窗口中看到运行结果
 - Read/Evaluate: 解释器读取python源程序的内容，执行这些python语句



内置函数print

- 数学函数：定义域中的元素按照确定的法则映射到值域中的元素
- 编程语言中的函数：接收相应的参数，**通过执行一系列的语句来完成相应的功能**，最后再返回结果

$$f(x) = \begin{cases} x^2 & x \geq 0 \\ -x & x < 0 \end{cases}$$

- 函数定义(define): def语句
- 函数调用: func(...)
 - func为要调用的函数名
 - 括号内为调用该函数所传递的参数，多个参数时，参数之间以逗号隔开
- 函数的3个基本要素：
 - 传递的参数
 - 要完成的功能
 - 返回值
- **print(...)** 调用print函数，将相应的值输出到标准输出（屏幕）上：
 - 支持可变长(0、1、2、3, ... 个) 的参数
 - 功能：将各个参数的值输出，参数之间以默认的空格(sep=' ') 隔开，最后输出默认的换行符(end='\n')
 - **返回None**

```
def f(x):  
    """上述数学函数f(x)的实现"""  
    if x >= 0:  
        return x ** 2  
    else:  
        return -x  
  
t = f(2)  
print(t)
```

```
>>> print('hello')  
hello  
>>> print("hello", "world")  
hello world  
>>> print("hello", "world", "!")  
hello world !  
>>> print()  
  
>>>
```

内置函数input

input(prompt=None): 等待用户输入, 返回用户输入的字符串

- 传递的参数:
 - 可传递0个参数, 默认prompt为None
 - 可传递1个参数, 此时首先输出prompt
- 功能: 输出提示符; 等待用户输入, 直到用户按**回车键(Enter)**结束, 构造一个字符串 (不包括回车) 并返回; 如果在输入回车之前, 按<Ctrl-C>, 则抛出异常KeyboardInterrupt
- 返回: 用户输入的**字符串** (不包括最后的回车)

函数的3个基本要素:

- 传递的参数
- 要完成的功能
- 返回值

```
>>> x = input()
45.7
>>> x
'45.7'
>>> x = input('Please input:')
Please input:3
>>> x
'3'
```

```
>>> x = 4'
SyntaxError: unterminated string literal (detected at line 1)
```

Please input: 提示符

3 用户输入的字符

Traceback: 出现异常(exception)之后解释器输出的错误信息

- 调用栈的信息 (哪一行调用哪个函数或者语句)
- 最后一行: 什么类型的异常以及原因

```
>>> x = input('N = ')
N = 2323Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyboardInterrupt
```

查看帮助

- 在交互式控制台，可以使用**help和dir函数**查看帮助信息

dir()	列出当前名字空间中的所有名字
dir(object)	列出object相关的属性和方法列表，其中下划线(_)开头的名字表示内部使用，如dir(str), dir('hello')
help(object)	查看对象object相关的帮助，比如help(print)查看函数print的相关帮助，help(3.14)查看浮点数支持的属性和方法, help(str)查看类型为str的字符串对象的属性和方法
help()	进入交互式的Help系统，键入quit退出帮助系统
help('topic')	查看交互式help系统的某个主题(topic) 相关的帮助，比如help('if')查看if语句语法

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'y']
>>> dir(str)
[... , 'swapcase', 'title', 'translate', 'upper', 'zfill']
>>> help('if')
```

```
...
    if_stmt ::= "if" expression ":" suite
               ( "elif" expression ":" suite ) *
               ["else" ":" suite]
```

```
>>> help(print)
```

Help on built-in function print in module builtins:

```
print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.
```

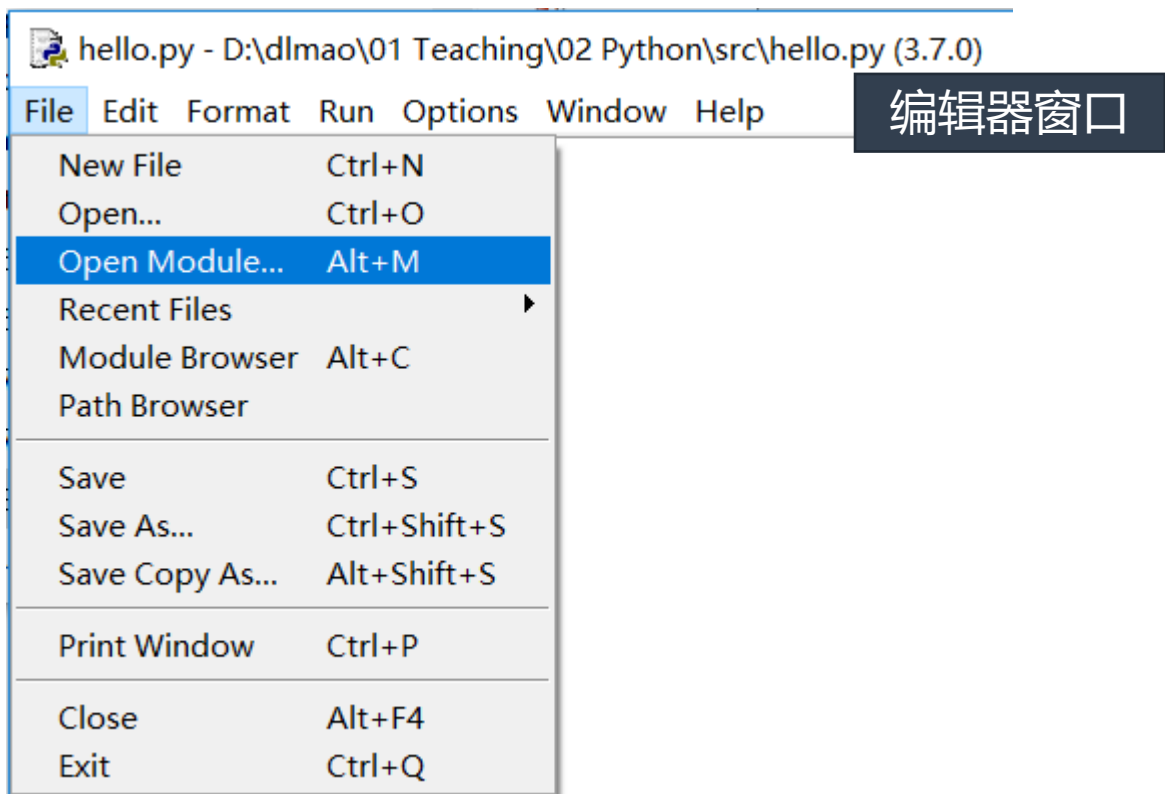
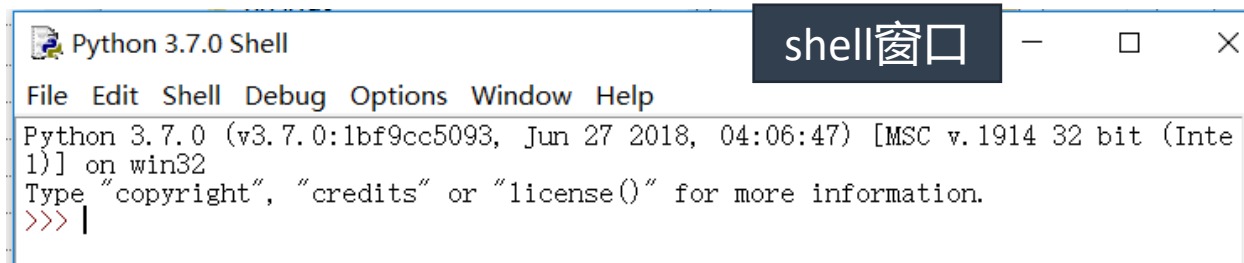
```
>>> help(str.upper)
```

Help on method descriptor upper:

```
upper(self, /) unbound builtins.str method
    Return a copy of the string converted to uppercase.
```

IDLE: 编写Python源程序(后缀名一般为.py)

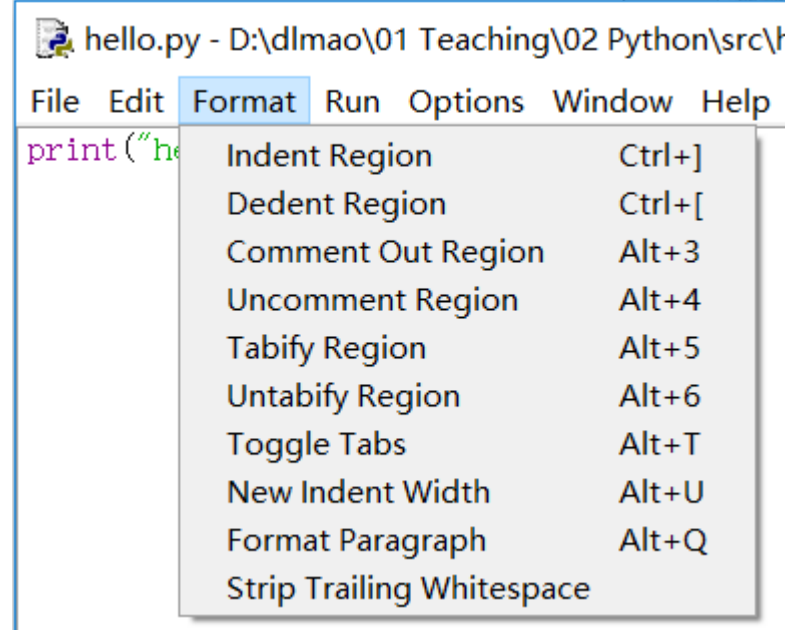
- 新建源程序: **File→New File** (Ctrl-N)创建一个新文件, 弹出新的编辑器窗口
- 打开已有的源程序
 - 菜单项**File→Open(Ctrl-O)**, 或者
 - 在windows系统中**资源管理器中鼠标右键弹出上下文菜单**, 选择Edit With IDLE
- **编辑器窗口:**
 - 在代码输入完毕后, **保存该文件**(File→Save, Ctrl-S), 选择要保存在哪个目录, 指定该源程序的文件名
 - Run子菜单, 如果未保存, 提醒保存后运行
 - Run→Python Shell: 切换到Shell窗口
 - **Run→Run Module(F5)运行**, 可以在Python Shell窗口中看到运行结果
 - Run→Run Customized, 类似于Run Module, 可以指定额外的参数
 - Run→Check Module仅仅检查编写的程序是否有语法错误, 并不运行



IDLE常用快捷键

- **撤销** (Ctrl+Z)、**全选** (Ctrl+A)、**复制** (Ctrl+C)、**粘贴** (Ctrl+V)、**剪切** (Ctrl+X)

快捷键	功能说明
Tab	智能缩进或者自动完成（输入前缀后会列出相关的关键字和属性）
Ctrl+]	缩进代码块(Indent Region)
Ctrl+[	取消代码块缩进(Dedent Region)
Alt+3	注释代码块(Comment Region)
Alt+4	取消代码块注释(Uncomment Region)



Format子菜单

- **Indent/Dedent Region: 缩进控制**
- **Comment/Uncomment Region: 注释控制**
- Strip Trailing Whitespace: 每行最后多余的空格移走

主要内容

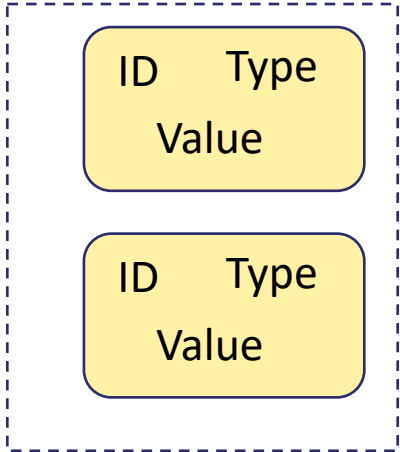
- 计算机概述
- 程序设计语言
- Python程序的运行
- **对象基础**
- 模块

数据类型和对象

(逻辑上的)对象空间

- 数据(值, value) 被保存在某种存储 (内存或外存) 中
 - 以二进制方式保存, 基本存储单元为字节
 - 104 → 01101000
 - 'h' → 01101000

id(object)	返回object的ID
type(object)	返回object的类型



- 类型: 比如整数、实数和字符串
 - 描述了一组相关值 (值的取值范围), 以及可以对这些值执行的一组操作
 - 有些程序语言(比如C, C++等) 要求程序员在代码中显式地声明数据的类型
 - python语言不要求声明数据类型
- 对象(object): Python中各种数据的抽象, 有**三个基本的属性**, 除此之外, 对象还可包括其他属性
 - 标识ID(identity): 对象一旦创建, 在其生命周期其**ID不再改变**, 可以看成该对象在内存中的地址 (cython实现)
 - 类型(type): 决定了对象可能取值的范围及支持的操作。也决定了在内存中如何组织该对象
 - 对象是类型的一个实例 (instance)
 - 对象的**类型不可变**, 即Python是一种强类型语言
 - 值(value): **值可以被改变或不可变**, 决定于其类型
 - 不可变(immutable): 有些对象一旦创建其值不可变, 比如数字类型、字符串、元组等
 - 可变(mutable): 对象的值可以改变, 比如列表、字典等

基本数据类型

- Literal(字面值/**字面量**): 表示某个内置对象类型的值, 在词法和语法分析时识别其类型
- 表达式**(expression): 一个值, 或者多个值通过各种运算符(operator, 包括函数调用等)运算之后的结果

$(3 * 4 + 5) * 6$

数值
类型

类型	描述	示例
int	整数	42, 20493, 0
float	实数(浮点数)	7.35, 4., 6.022e23
complex	复数	1+2j, 3.4-1j
bool	布尔	True, False
NoneType	空类型	None
str	字符串	"Hello.", 'abc 1 2', ''
list	列表	[1, 2, 3]
tuple	元组	(1, 2, 3)
dict	字典	{'name':'tony', 'age':22}
set	集合	{1, 2, 3}

```
>>> id(3.14)
2078394753840

>>> type(3.14)
<class 'float' >

>>> type(3)
<class 'int'>

>>> type(True)
<class 'bool'>

>>> type(print)
<class 'builtin_function_or_method'>

>>> import math
>>> type(math)
<class 'module'>

>>> type(math.sin)
<class 'builtin_function_or_method'>
```

- NoneType类型仅仅有一个值, 即None, 常用于函数返回值
- 函数、模块、类也是对象

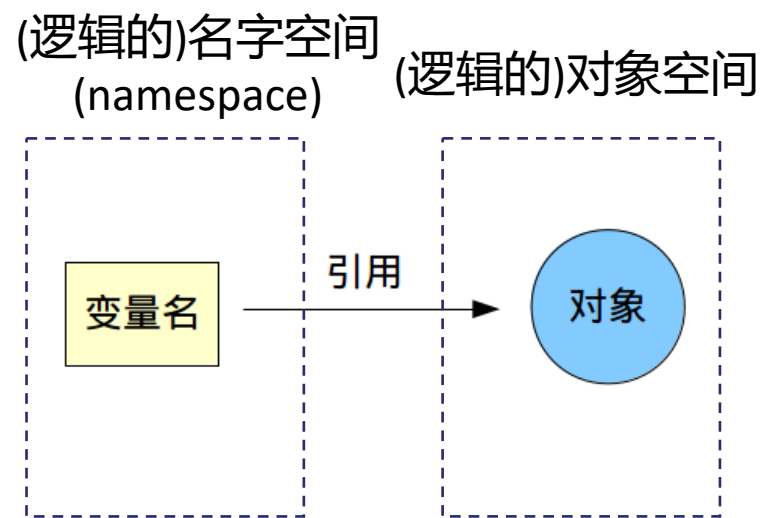
变量和赋值语句

python注释: #字符后的内容不会被解释器解释

```
a = 3 + 4
a = a + 3
b = 3.14
c = d + 8 # NameError
```

- 对象: Python中各种数据的抽象
- 表达式(expression): 各个对象通过各种运算符(operator)运算之后的结果 $(3 * 4 + 5) * 6$
- **变量(variable):** 通过名字表示对于某个对象的引用或指针 (reference, pointer)
 - 变量有一个名字, 称为变量名
 - 通过**赋值语句(assignment statement)**给变量赋值, 类似于给对象取一个别名(alias)
 - $\text{variable} = \text{expression}$ (LHS = RHS, left-hand side/right-hand side)
 - 首先计算RHS的表达式值 (对象), 然后LHS的变量指向该对象 (即值赋给变量)
 - **变量出现在LHS (Left Hand Side), 给该变量赋值, 即保存RHS所对应对象的引用 (地址)**
 - **变量出现在RHS (Right Hand Side) 处时表示引用该变量所指向对象的值**
 - 变量在使用前 (出现在表达式) 必须有定义 (赋值)
 - 变量在赋值之前**无需声明其类型**, 变量的类型为所指向对象的类型
- Python是一种**动态类型**语言
 - 变量的类型是可以随时变化的
 - 对象的类型本身是不会变的
- 名字空间通过字典dict实现, 字典保存的是key:value对, key为变量的名字, 而value为对应的对象

Namespaces are one honking great idea -
- let's do more of those!
The Zen of Python (import this)



变量和赋值语句

- 变量表示某个对象的引用，允许多个变量指向同一个对象
- $a = a + 3$ ，变量 a 指向了原来变量 a 的值与3相加后的另一个对象
- $b = a$ 相当于给 a 所指向的对象取了另外一个**别名** b

```
>>> a = 3 + 4
```

```
>>> id(a)
```

```
1688352877040
```

```
>>> a = a + 3
```

```
>>> id(a)
```

```
1688352877136
```

```
>>> b = a
```

```
>>> id(b)
```

```
1688352877136
```

```
>>> b = 3.14
```

```
>>> id(b)
```

```
2078394754544
```

```
>>> c = d + 8
```

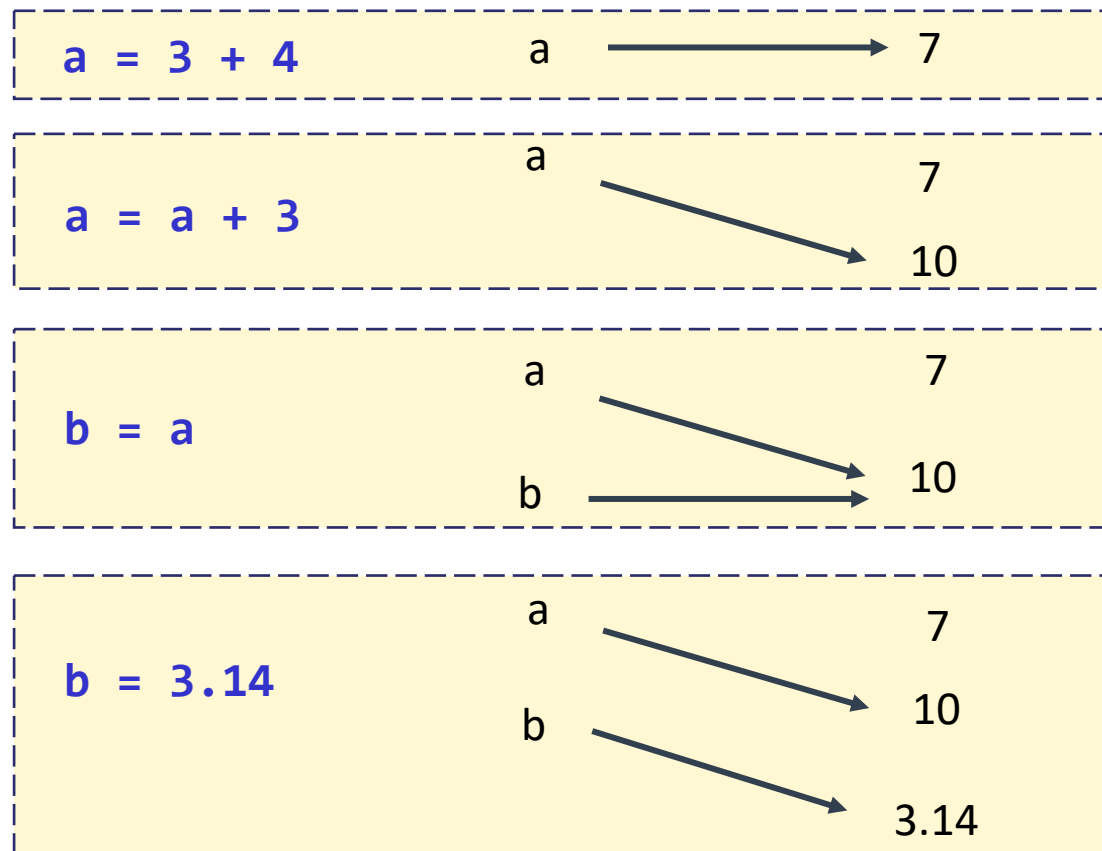
```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
NameError: name 'd' is not defined. Did you mean: 'id'?
```

名字空间
(namespace)

对象空间



复合赋值与链式赋值

- 复合赋值 (augmented assignment):

var op= expression , op为算术运算符(+ - * / // % **)或位运算符(& | ^)

- 注意运算符和=之间没有空格
- 简化代码, 精炼程序
- 对于不可变对象, 相当于 **var = var op expression**

total += item 等价于 total = total + item

total += item # 语法错误SyntaxError: invalid syntax

total *= item 等价于 total = total * item

total **= item 等价于 total = total ** item

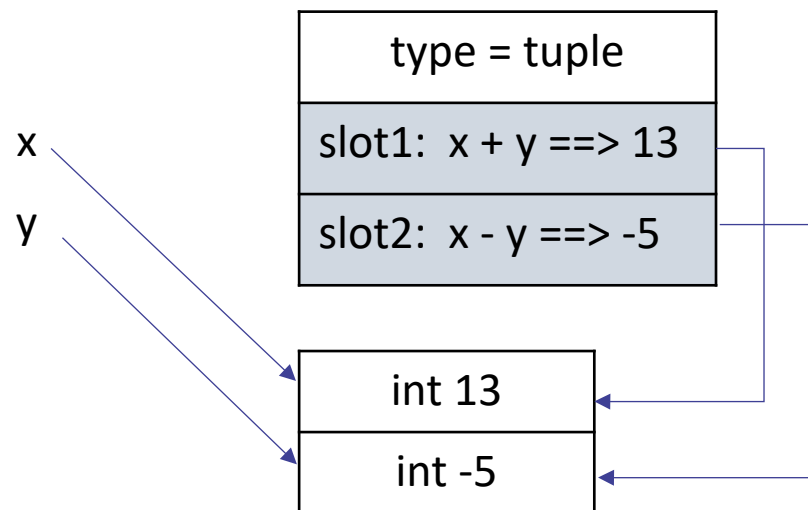
- 如果var保存的为**可变类型**的列表对象, 复合赋值为IN-PLACE计算, 提高效率
 - 链式赋值 (chained assignment): **var1 = var2 = ... = expression**
 - 右结合, 即首先 var2 = ; var1 = var2
- x = y = 123 等价于 y = 123; x = y

- ✓ Python一般一行一个语句, 但也可在同一行中放置多条语句, 语句之间使用分号 “;” 分割, 但为可读起见, 不建议

序列解包与赋值语句（教材中的多变量赋值）

序列解包与赋值语句结合：var1, var2 = value1, value2

- 首先计算value1, value2，构造一个tuple对象（一种可迭代对象）
- 右边的可迭代对象展开，按照顺序，将右边的各个值与左边的多个变量绑定（赋值）
- x, y = y, x 实现了两个变量x,y对调的功能



```
>>> x, y = 4, 9
>>> x, y = x + y, x - y
>>> print(x, y)
13 -5

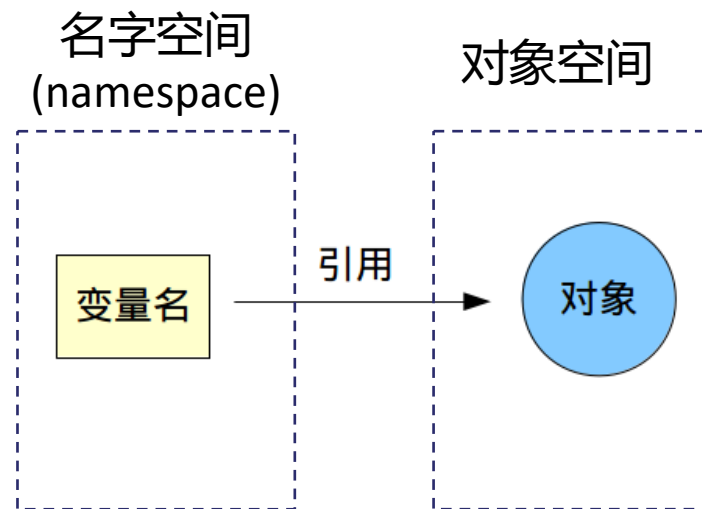
>>> x, y = y, x
```

del语句

del var

del var1, var2

- 将变量var与原有对象解绑，从相应的名字空间中删除
- 再访问var时会抛出NameError异常



```
>>> x = 4
>>> x
4
>>> del x
>>> x
Traceback (most recent call last):
  File "<pyshell#60>", line 1, in <module>
    x
NameError: name 'x' is not defined
```

变量名

- 变量的名字必须符合**标识符(identifier)**的语法
- 标识符必须**以字母或下划线**开头，其后的字符可以是**字母、下划线或数字**
 - 字母或数字：**unicode字符集**中的字母或数字类型的字符
 - 变量名中不能有空格及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等）
 - 非法的标识符： 99var It'OK first-class
 - 尽管中文字母可以作为标识符，但不建议使用。比如 年龄
- 标识符长度至少为1
- 标识符对英文字母的**大小写敏感**，例如student和Student是不同的变量
- 一些特殊的标识符保留为Python**关键字(keyword)**，不能用作变量名

```
>>> help('keywords')
```

```
Here is a list of the Python keywords. Enter any keyword to get more help.
```

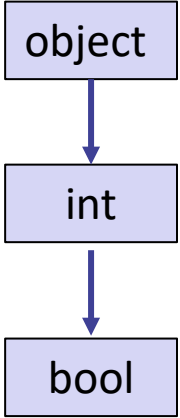
False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

变量名

- 不建议使用系统内置的类型名或函数名（比如int, input, print等）以及通过import语句导入的模块名等作为变量名，否则原来的对象就无法通过该名字访问了
- 建议使用**有意义的名字**，比如total、keys等
 - 单字母变量名(如i, j, k)仅用在较短的循环等结构
 - 如果变量名与内置的对象名字重复，可在最后添加下划线，比如sum_
- 避免使用小写字母l和大写字母0
- 以**下划线开头的变量**一般表示在某个范围中内部（临时）使用的变量
 - python标准控制台中使用变量_，记录最近的表达式计算的结果
 - python内部使用一些以双下划线开始和结束的dunder名字（double underline），具有特殊的含义
 - __name__, __doc__, __file__等
- 变量、函数、方法、模块等：建议使用**小写字母**，多个单词时以下划线隔开，lower_case_with_underscores
- 常量（保存的值不变）：建议使用大写字母，多个单词之间以下划线隔开：HOURS_PER_DAY
- 类(Class)：建议驼峰命名或Pascal命名法
 - 一个或多个单词，每个单词首字母大写：Model, PythonClass

类型、等值与等同

注意instance(obj, cls) 与type(obj) == cls的区别



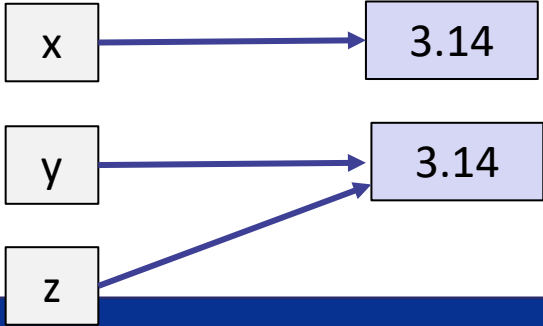
```
>>> x = True
>>> isinstance(x, bool)
True
>>> isinstance(x, int)
True
>>> isinstance(x, float)
False
```

```
>>> x = 3.14
>>> y = 3.14
>>> type(x)
<class 'float'>
>>> x == y
True
>>> x != y
False
>>> x is y
False
>>> x is not y
True
>>> z = y
>>> y is z
True
```

```
# mro: Method Resolution Order
>>> bool.__mro__
(<class 'bool'>, <class 'int'>, <class 'object'>)
```

id(object)	返回object的ID
type(object)	返回object的类型
isinstance(obj, cls)	测试对象obj是否为指定类型cls或其各级子类型的实例，返回True或False, cls可为int/float/str等。第2个参数cls也可以是多个类型组成的元组
obj1 == obj2 obj1 != obj2	判断obj1和obj2的值是否相等(或不等)，结果为True或False
obj1 is obj2 obj1 is not obj2	判断obj1和obj2是否同一（或不是同一）对象, 结果为True或False 等价于id(obj1) == id(obj2) 或 id(obj1) != id(obj2)

```
# 第2个参数也可可是多个类型组成的元组
>>> isinstance(4, (int, float))
True
>>> isinstance(4.5, (int, float))
True
```



主要内容

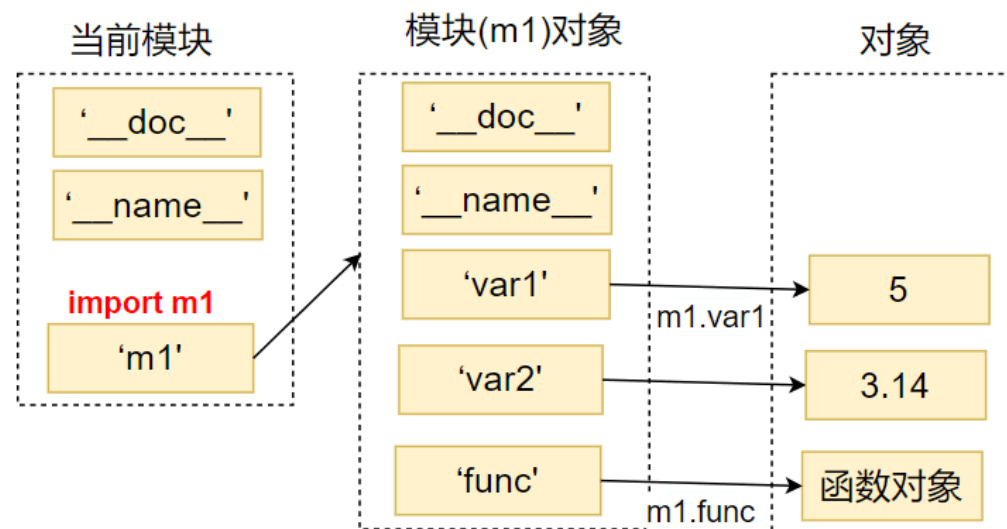
- 程序设计语言
- Python程序的运行
- 对象基础
- **模块**

模块

- 模块(module): 相互之间有联系的函数以及变量组织在一起, 放到同一个Python源文件 (.py)
 - 不同模块属于不同的**名字空间**, 可以避免函数名和变量名冲突
 - 提高了代码的可维护性和可重用性
- 如何使用模块? `import module [as alias]` # `import math`
 - 如果之前没有import, 寻找模块源文件, **加载并执行**模块中的代码, 构造模块对象
 - 在**当前名字空间引入名字**指向模块对象。 # 变量`math`指向加载的模块对象
- 如何访问模块(名字空间)中的对象(变量和函数)?
 - 在模块中定义的变量和函数保存在模块对象中, 作为其属性
 - 模块中的变量可通过模块对象的属性(attribute)
 - `obj.attr`: `obj`为指定的对象, 而`attr`为对象的属性名
 - 模块中的函数可通过模块对象的方法(method)访问
 - 特殊类型的属性**, 其保存的是可调用的对象(类似函数)
 - `obj.method`: `method`为对象的方法名
- (qualified)**限定名**: 通过dot路径限定其名字所在空间
- (unqualified)**未限定名**: 前面没有dot路径, 表示当前名字空间
 - `m1.var1`: `m1`为未限定名, `var1`为限定名
- 模块和模块之间的联系是通过import语句来建立的
 - 未限定的名字表示**当前模块的名字**, 其他模块中的名字通过import语句得到模块对象来限定访问

```
# m1.py
var1 = 5
var2 = 3.14
def func():
    return 0
```

```
# m2.py
import m1
var1 = 0
print(m1.var1)
print(m1.func())
```



import 语句

`import module [as alias]` 加载模块，在**当前名字空间引入某些名字**

- `import module`: 加载模块`module`，当前名字空间中的变量`module`指向加载的模块对象
- `import module as alias`: 加载模块`module`，变量`alias`指向加载的模块对象
- 多个`import`语句可合并在一起，表示加载多个模块，比如`import sys, math as math2`。但不建议
- `dir(module)`可以查看模块对象`module`中的属性。如果调用不带参数的`dir()`函数，则返回当前名字空间内的所有名字（前面定义的变量）
- `help(obj)`函数查看对象（如果传递参数为模块对象，则为模块）或函数（如果传递参数为函数）的使用帮助
- 内置模块**`builtins`**定义了许多内置的类型（`int/float`等）和内置函数（`print/input`）等
- 解释器执行了下述语句：
`import builtins as __builtins__`
- 调用的内置函数都来自于**`__builtins__`**所指向的（模块）对象的名字空间（**内置名字空间**）

```
>>> import math
>>> math.sin(3)
0.1411200080598672
>>> import math as math2
>>> math2.sin(3)
0.1411200080598672
```

```
>>> dir(math)
...
>>> help(math)
...
>>> help(math.sin)
...
>>> dir(__builtins__)
```

import语句

`from module import obj [as alias]`

- 仅从module中**导入名字为obj的对象**，当前名字空间引入变量obj或alias，指向该导入的对象
- 访问该对象时**不再需要**包括模块名，可以减少查询次数，提高执行速度

`from module import *` **拓展：该语句不允许在函数体内使用**

- 从模块中导入所有的对象（有些特殊的名字除外）到当前模块的名字空间
- 多个模块中有同样的对象名时造成混乱，因此谨慎使用
- 特殊的名字：
 - 如果待导入的模块(module) 中定义了__all__变量，则在使用from module import *时，只有该变量中列出的名字才可以被其他模块导入
 - 如果没有__all__变量，则以**单下划线开始的变量(比如_xxx)**被排除在外

```
__all__ = [  
    'abc', 'def'  
]
```

```
>>> from math import sin  
>>> sin(3)  
0.1411200080598672  
>>> from math import sin as f, cos  
>>> f(3)  
0.1411200080598672  
>>> cos(0)  
1.0  
>>> from math import *
```

导入模块

- 如果**第一次**导入，则寻找模块源程序，**加载(也就是执行)**模块，保存**模块对象**
 - 从**sys.path**给出的位置或目录列表中查找，找到第1个名字为模块名的源程序，**执行该源程序**
 - 已经导入的模块保存在字典对象**sys.modules**，key为模块名，而value为新创建的模块对象
- 可以多次调用import，但是**只加载执行一次**
 - 如果该模块名已经出现在sys.modules，则找到对应的模块对象，添加相应的对象引用
- **sys.path**：列表对象，可通过列表对象的insert,append等方法增加其他目录，扩展模块的搜索范围

```
>>> import sys
>>> sys.path
['', 'D:\\Software\\Python39\\python39.zip', 'D:\\Software\\Python39\\DLLs',
'D:\\Software\\Python39\\lib', 'D:\\Software\\Python39', 'D:\\Software\\Python39\\lib\\site-packages']
```

可以看到首先从当前目录中查找，最后是安装目录下的site-packages

Python之禅 (The Zen of Python)

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

```
>>> import this
```

—— Tim Peters

优美胜于丑陋。

显式胜于隐式。

简单胜于复杂。

复杂胜于难懂。

扁平胜于嵌套。

分散胜于密集。

可读性应当被重视。

尽管实用性会打败纯粹性，特例也不能凌驾于规则之上。

除非明确地使其沉默，错误永远不应该默默地溜走。

面对不明确的定义，拒绝猜测的诱惑。

用一种方法，最好只有一种方法来做一件事。

虽然一开始这种方法并不是显而易见的，但谁叫你不是

Python之父呢。

做比不做好，但立马去做有时还不如不做。

如果实现很难说明，那它是个坏想法。

如果实现容易解释，那它有可能是个好想法。

命名空间是个绝妙的想法，让我们多多地使用它们吧！

优雅、明确、简单

__name__

除了在源程序中定义的名字外，python解释器还会（在模块真正执行之前）定义一些内部的名字
模块一般有两种方式执行

- 在IDLE环境中以Run Module方式运行，即该模块为主入口程序，也称为**脚本(script)方式**运行
 - 主入口(脚本)方式运行时，__name__被设置为字符串 "__main__"
 - 每个模块经常会包含测试代码以测试其所定义的函数，这些代码以脚本方式运行时**应该执行**
- 作为**模块**被导入(即**被其他模块import**)，如果为第1次导入，则执行该模块源程序中的代码
 - 以import方式执行的模块中自动增加的名字中，__name__被设置为该模块的模块名
 - 模块中经常包含的测试代码在import时**不应该执行**
- sys.modules**给出模块名字与模块对象的映射，即__name__:模块对象

在模块最后，一般包含：

```
if __name__ == '__main__':  
    #以脚本方式运行时才会执行的代码
```

__name__	模块的名字，以脚本方式运行时设置为" __main__ "，import方式运行时设置为 模块的文件名 (不包括.py)
__file__	模块源程序的完整路径名
__doc__	文档字符串，如果模块的第一个语句为字符串字面量，则该字符串作为文档字符串保存在__doc__中

```
pi = 3.14  
print('loading...', __file__)  
if __name__ == '__main__':  
    print("Hello World")
```

main.py

Python程序结构（推荐，非强制）

```
import ... .. # 导入外部模块
import ... ..

def my_func(...): # 自定义函数
    .....
    .....

def main(): # 主流程，亦是自定义函数，可以是其他函数名
    .....
    ..... # 调用 my_func(.) 等
    .....

if __name__ == "__main__" :
    main() # 执行主流程
```

python命令行使用

- 首先进入操作系统的命令行接口：
 - Windows的命令提示符应用(cmd.exe)
 - 苹果操作系统的终端应用(terminal)
- `python -h` 查看python命令的帮助
 - `-c cmd`: python解释器执行cmd命令后退出
 - `-m mod`: python解释器以脚本方式执行模块mod
 - `-i`: python解释器执行之后仍然在交互模式

```
C:\Users\dlmao>python -h
usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ...
...
C:\Users\dlmao>python -c "import sys; print(sys.version)"
3.12.2 (tags/v3.12.2:6abddd9, Feb 6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)]
C:\Users\dlmao>python -m this
The Zen of Python, by Tim Peters
...
C:\Users\dlmao>python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...

C:\Users\dlmao>python hello.py
...
C:\Users\dlmao>python -i hello.py
...
>>>
```

pip: 第三方库的安装

- 访问<https://pypi.python.org/pypi>来查找相应的package
- 在操作系统的命令行应用程序中执行 **pip install package_name**, 或者 **python -m pip install package_name** 安装相应的扩展包
- pip list: 列出目前已经安装的package
- pip show package: 查看package的相关情况
- pip install package: 安装package
 - pip install --upgrade package 将package升级到最新版本
 - pip install -r requirements.txt: 安装requirements.txt文件中描述的指定版本的package
- pip uninstall package: 卸载package
- pip freeze: 按照requirements.txt文件的格式, 输出目前安装的第三方package(不包括pip等自带的package)以及版本号

```
$ pip install autopep8
Collecting autopep8
...
Collecting pycodestyle>=2.10.0 (from autopep8)
...
stalling collected packages: pycodestyle, autopep8
Successfully installed autopep8-2.0.4 pycodestyle-2.11.0
$ pip list
Package      Version
-----
autopep8     2.0.4
pip          23.2.1
pycodestyle  2.11.0
setuptools   65.5.0
$ pip freeze
autopep8==2.0.4
pycodestyle==2.11.0
$ pip freeze > requires.txt
$ pip uninstall autopep8
...
$ pip install -r requires.txt
...
```

课堂练习

下面给出了若干名字，其中可以作为变量名的有几个？（ ）

(1) _ (2) 变量 (3) tom's (4) True (5) 2apples

A: 1

B: 2

C: 3

D: 4

E: 5

课堂练习

- 分别运行main.py和main2.py, 检查程序的输出

main.py

```
pi = 3.14
print('loading...', __file__)
if __name__ == '__main__':
    print("Hello World")
```

main2.py

```
import main
print('loading...', __file__)
if __name__ == '__main__':
    print("Hello world 2")
```

课堂练习

- 第1章 习题4：找出下面代码中的错误

```
# 显示球体体积  
r = 2.6  
print(" 半径=", r)  
V = (4/3) $\pi$ r3  
    print(" 体积=", V)
```

编程题

第1章习题5： 某国人口普查局基于如下假设来预测人口：

- 每7秒1人出生
- 每13秒1人死亡
- 每45秒1人移入
- 每79秒1人移出

编写程序显示接下来5 年每一年的人口。假设当前的人口数是3120324986， 每年有365 天。

其他Slides

缓存复用（不要求）

无法改变值的不可变(immutable)对象可以重用已有的具有相同值的对象

- 牺牲（缓存查找）时间换取空间的优化策略
- [-5, 256]的整数，如果缓存中已有相同值的整数对象，则重用该对象
- 较短的字符串也采用类似的设计
- True, False, None也是如此。在一个对象x是否为None时，建议： `x is None`，而不是`x == None`

```
>>> x = 100
>>> y = 100
>>> x == y
True
>>> x is y
True
>>> id(x) == id(y)
True
```

